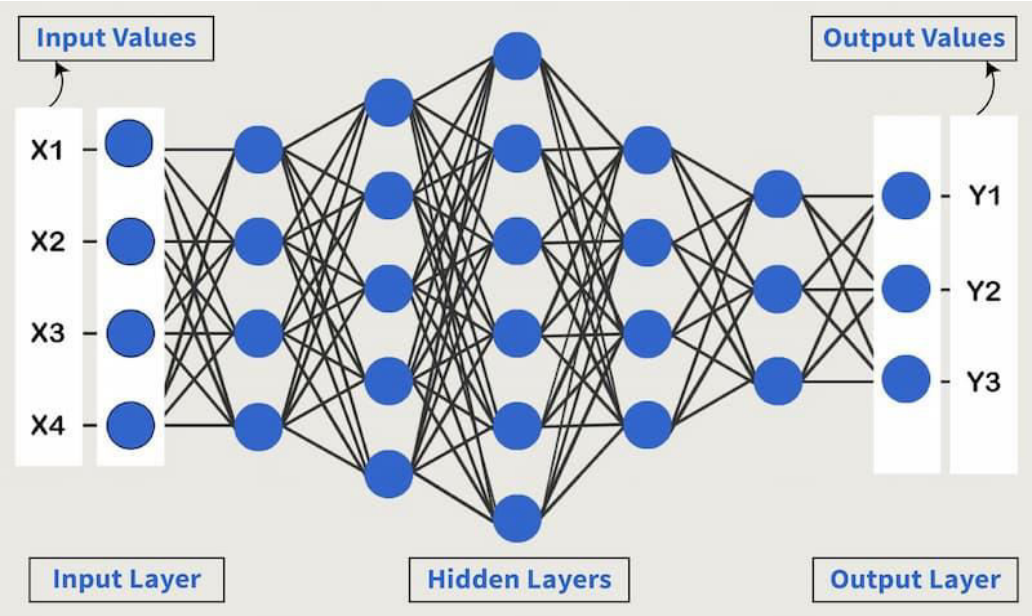
1. Introduction to Neural Networks:

- **Concept:** Neural networks are computational models inspired by the biological neural networks in our brains. They are designed to learn complex patterns from data. At their core, they consist of interconnected nodes (neurons) organized in layers.
- **Resources:**
  - **Stanford CS231n (Convolutional Neural Networks for Visual Recognition):** While focused on CNNs, the initial lectures provide an excellent foundation in neural networks in general. (Search for CS231n online)
  - **Michael Nielsen's "Neural Networks and Deep Learning":** A free online book with a very accessible introduction.
  - 

2. Biological and Artificial Neuron:

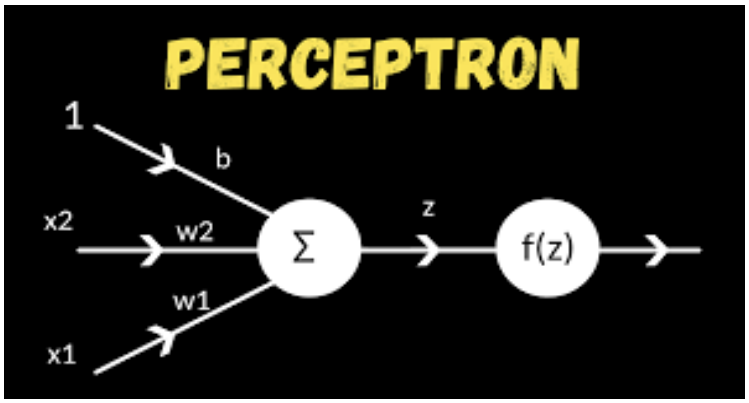
- **Biological Neuron:** A biological neuron receives signals through dendrites, processes them in the cell body, and transmits the output along the axon. Synapses are connections between neurons where signals are passed using neurotransmitters.
- **Artificial Neuron (Perceptron):** A simplified mathematical model of a biological neuron. It receives inputs, multiplies them by weights, sums them, and applies an activation function to produce an output.
- **Resources:**
  - **Wikipedia:** Search for "Biological neuron" and "Artificial neuron."
  - **Textbooks on Computational Neuroscience:** These often have good explanations of the biological inspiration.

3. Introduction to Perceptron:

A perceptron is the simplest type of artificial neural network used for binary classification. It consists of:

- **Input Layer**: Accepts input features.
- **Weights**: Assigns importance to input features.

- **Summation Function**: Computes the weighted sum of inputs.
- **Activation Function**: Determines whether the neuron is activated.



Concept: The perceptron is the simplest type of neural network, a single-layer network. It's a linear classifier.

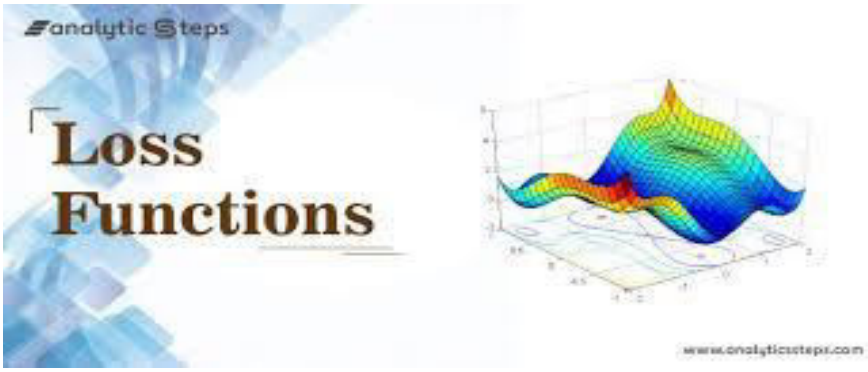
- **Perceptron Learning Rule:** The perceptron learning rule adjusts the weights based on the error between the predicted and actual outputs.
- **Drawbacks:** Perceptrons can only classify linearly separable data. They cannot handle complex patterns.

4. Multilayer Perceptron (MLP):

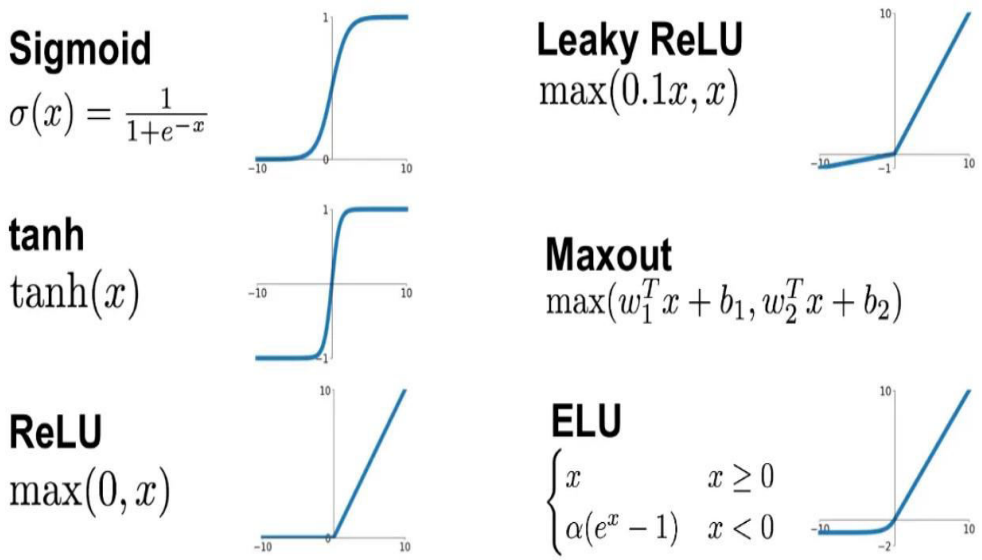
- **Concept:** MLPs have multiple layers (at least one hidden layer) of neurons. This allows them to learn non-linear relationships and classify more complex data.
- **Loss Function:** Measures the error between the network's predictions and the actual values. Common loss functions include mean squared error (MSE) and cross-entropy.
- **Resources:**
  - **"Deep Learning" by Goodfellow, Bengio, and Courville:** A comprehensive textbook.

5. Loss Functions

Loss functions measure the difference between predicted and actual outputs. Common loss functions include:

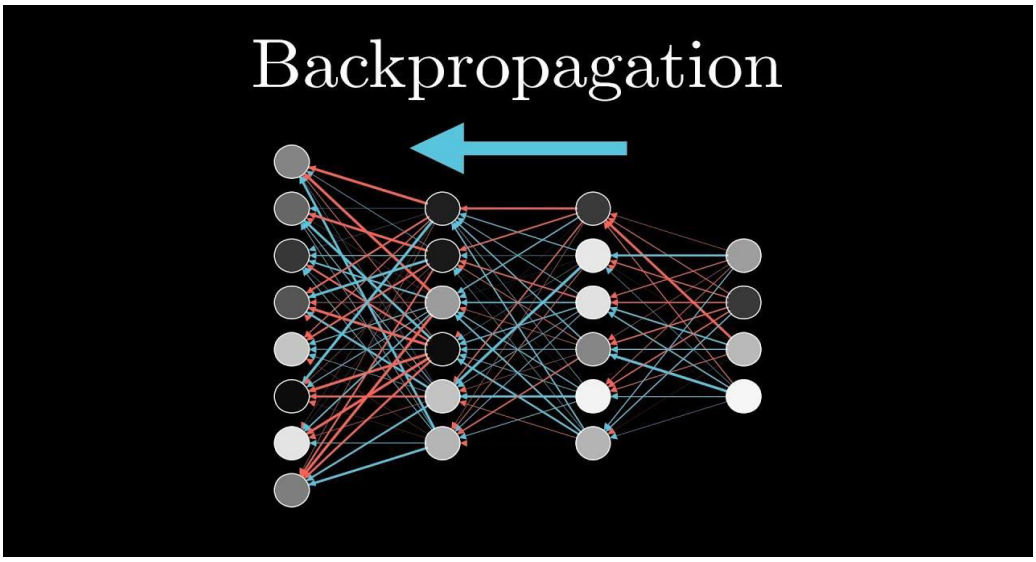
- **Mean Squared Error (MSE)**: Used for regression tasks.
- **Cross-Entropy Loss**: Used for classification tasks.
- 

6. Neural Network Activation Functions:

- **Concept:** Activation functions introduce non-linearity into the network, enabling it to learn complex patterns. Examples:
  - **Sigmoid:** Outputs values between 0 and 1.
  - **Tanh:** Outputs values between -1 and 1.
  - **ReLU (Rectified Linear Unit):** Outputs 0 for negative inputs and the input value for positive inputs. ReLU and its variants are very popular.
- **Resources:**
  - **Visualizations:** Search for "activation functions visualization" to see how they behave.
  - 

7. Training MLP: Backpropagation:

- **Concept:** Backpropagation is the algorithm used to train MLPs. It calculates the gradients of the loss function with respect to the weights and uses these gradients to update the weights in the direction that minimizes the loss.
- **Cost Function:** Synonymous with the loss function.
- **Gradient Descent:** An optimization algorithm used to minimize the cost function by iteratively updating the weights in the direction of the negative gradient.
- **Vanishing and Exploding Gradient Problem:** During backpropagation, gradients can become very small (vanishing) or very large (exploding), making training difficult. Techniques like ReLU activation and batch normalization can help mitigate this.
- **Resources:**
  - **CS231n:** Excellent explanations of backpropagation.
  - **Online tutorials:** Search for "backpropagation explained."



8. Introduction to PyTorch

Concept: PyTorch is a popular open-source deep learning framework. It provides tools for building, training, and deploying neural networks, Key features:

- Tensor computations similar to NumPy.
- Dynamic computation graphs.
- GPU acceleration.
- In-built modules for deep learning.

9. Regularization

Regularization prevents overfitting by adding constraints to model training:

- **L1 Regularization (Lasso)**: Adds absolute weight penalties.
- **L2 Regularization (Ridge)**: Adds squared weight penalties.
- **Dropout**: Randomly drops neurons to reduce overfitting.

10. Optimizers

Optimizers adjust model weights to minimize the loss function. Popular optimizers include:

- **SGD (Stochastic Gradient Descent)**

- **Adam (Adaptive Moment Estimation)**
- **RMSprop (Root Mean Square Propagation)**

11. Hyperparameter Tuning

Hyperparameters control the learning process and must be optimized for best performance. Common tuning techniques:

- **Grid Search**: Tries all possible hyperparameter combinations.
- **Random Search**: Randomly selects hyperparameters.
- **Bayesian Optimization**: Uses probabilistic models for optimization.

Resources:

"Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow":

Interview Questions:

I. Foundational Concepts:

1. What is the fundamental difference between a biological neuron and an artificial neuron?
2. Explain the perceptron learning rule in simple terms.
3. What are the key limitations of a perceptron?
4. Why are Multilayer Perceptrons (MLPs) more powerful than single-layer perceptrons?
5. Describe the role of the activation function in a neural network. Give examples.
6. What is the purpose of the loss function (or cost function)?
7. Explain the concept of gradient descent and its role in training neural networks.
8. What is backpropagation, and how does it work?
9. What is the difference between supervised and unsupervised learning? (A broader ML question, but relevant)

II. Activation Functions & Backpropagation:

10. Why are non-linear activation functions necessary in deep learning?
11. Compare and contrast sigmoid, tanh, and ReLU activation functions. What are their advantages and disadvantages?
12. Explain the vanishing and exploding gradient problems. How can they be mitigated?
13. What is the chain rule, and how is it used in backpropagation?

III. Optimization & Regularization:

14. Explain the difference between batch gradient descent, stochastic gradient descent, and mini-batch gradient descent. Why is mini-batch gradient descent often preferred?
15. What is the purpose of regularization in neural networks?
16. Describe L1 and L2 regularization. What are their effects on the weights?

17. Explain dropout regularization. How does it prevent overfitting?
18. What are some common optimization algorithms used in deep learning? (e.g., Adam, RMSprop)
19. What is the difference between a hyperparameter and a model parameter?
20. How would you approach tuning the hyperparameters of a neural network?

IV. PyTorch & Practical Questions:

21. What are tensors in PyTorch, and why are they important?
22. How do you define a custom neural network architecture in PyTorch?
23. Explain the difference between model.train() and model.eval() in PyTorch.
24. How do you load and preprocess data for deep learning in PyTorch?
25. Describe the typical workflow for training a neural network in PyTorch.
26. What are some common metrics used to evaluate the performance of a classification model? (e.g., accuracy, precision, recall, F1-score)
27. How would you handle imbalanced datasets in a classification problem?
28. Let's say your model is overfitting. What steps would you take to address this issue?
29. You're training a neural network, and the loss is not decreasing. What are some possible reasons and how would you troubleshoot them?
30. How would you approach a new deep learning problem (e.g., image classification, natural language processing)? Describe the steps you would take.
31. What is transfer learning, and why is it useful?
32. Explain the concept of convolutional layers in CNNs and their role in image processing.
33. What are recurrent neural networks (RNNs), and what type of data are they best suited for?
34. Explain the difference between a CNN and an RNN.
35. What are some challenges you anticipate when deploying a deep learning model to production?

Key Learning Path:

1. **Start with the basics:** Understand biological neurons, artificial neurons, and perceptrons.
2. **Move to MLPs:** Learn about hidden layers, activation functions, and why MLPs are more powerful than perceptrons.
3. **Master backpropagation:** This is crucial for understanding how neural networks learn.
4. **Explore PyTorch:** Get hands-on experience by implementing neural networks in PyTorch.
5. **Dive into advanced concepts:** Learn about regularization, optimizers, and hyperparameter tuning to improve your models.

DEEP
LEARNING